

```
In [104... import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from ucimlrepo import fetch_ucirepo
```

```
In [105... # 1. Load dataset directly from UCI repository
heart_disease = fetch_ucirepo(id=45)
```

```
In [106... # 2. Extract features (X)
df = heart_disease.data.features.copy()
```

```
In [107... # 3. Add target column (y) to the dataframe
df['target'] = heart_disease.data.targets
```

```
In [108... # 4. Show first 5 rows
print(df.head())
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	sl
ope \											
0	63	1	1	145	233	1	2	150	0	2.3	
3											
1	67	1	4	160	286	0	2	108	1	1.5	
2											
2	67	1	4	120	229	0	2	129	1	2.6	
2											
3	37	1	3	130	250	0	0	187	0	3.5	
3											
4	41	0	2	130	204	0	2	172	0	1.4	
1											

	ca	thal	target
0	0.0	6.0	0
1	3.0	3.0	2
2	2.0	7.0	1
3	0.0	3.0	0
4	0.0	3.0	0

```
In [109... # 5. Check dataset info
print(df.info())
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   age         303 non-null   int64
 1   sex         303 non-null   int64
 2   cp          303 non-null   int64
 3   trestbps    303 non-null   int64
 4   chol        303 non-null   int64
 5   fbs         303 non-null   int64
 6   restecg     303 non-null   int64
 7   thalach     303 non-null   int64
 8   exang       303 non-null   int64
 9   oldpeak     303 non-null   float64
10  slope       303 non-null   int64
11  ca          299 non-null   float64
12  thal        301 non-null   float64
13  target      303 non-null   int64
dtypes: float64(3), int64(11)
memory usage: 33.3 KB
None

```

```

In [110]: # Check missing values
print(df.isnull().sum())
df = df.fillna(df.median(numeric_only=True))
print(df.isnull().sum())

```

```

age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          4
thal        2
target      0
dtype: int64
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64

```

```
In [111... # 6. Check unique values in target
print(df['target'].unique())
```

```
[0 2 1 3 4]
```

```
In [112... # 7. Convert target into binary:
# 0 = no disease, 1 = disease present (values 1-4)
# This simplifies the problem and makes it suitable for classification mo
df['target_binary'] = df['target'].apply(lambda x: 0 if x == 0 else 1)
```

```
In [113... # 8. Verify transformation
print(df[['target', 'target_binary']].head(10))
```

	target	target_binary
0	0	0
1	2	1
2	1	1
3	0	0
4	0	0
5	0	0
6	3	1
7	0	0
8	2	1
9	1	1

```
In [114... # 9. Check class distribution
print(df['target_binary'].value_counts())
```

```
target_binary
0    164
1    139
Name: count, dtype: int64
```

Target Transformation

The original dataset contains 5 classes (0–4), representing the severity of heart disease.

For this project, we converted it into a binary classification problem:

- 0 means no heart disease
- 1 means presence of heart disease (values 1–4)

This simplifies the modelling process and aligns with the project objective.

Feature Selection and Train-Test Split

```
In [115... from sklearn.model_selection import train_test_split
```

```
In [116... X = df.drop(columns=['target', 'target_binary'])
y = df['target_binary']
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

```
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

```
X_train shape: (242, 13)
X_test shape: (61, 13)
y_train shape: (242,)
y_test shape: (61,)
```

Feature Scaling

In this section, we standardize the features using StandardScaler.

Scaling is important for models like KNN and SVM, as they are sensitive to the magnitude of feature values.

```
In [117... from sklearn.preprocessing import StandardScaler
```

```
In [118... scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
print(X_train_scaled[:5])
```

```
[[-2.83850353  0.72250438 -1.10468954 -0.12598175 -0.86414249 -0.39735971
  1.01249171  2.31447034 -0.71589105 -0.87357299 -0.96343165 -0.71586852
 -0.86916075]
 [ 0.24135234  0.72250438 -0.09205746  0.97465301 -2.48363748  2.51661148
 -0.99589349  1.02124161 -0.71589105 -0.70485418 -0.96343165  0.40181007
 1.20824075]
 [ 1.56129057  0.72250438 -2.11732162  1.52497038 -0.2412598  2.51661148
 1.01249171 -0.85171034 -0.71589105 -0.78921359  0.65566876  0.40181007
 -0.86916075]
 [ 1.12131116 -1.38407465 -0.09205746  1.52497038  2.3748475  -0.39735971
 1.01249171  0.04017154 -0.71589105 -0.19869777 -0.96343165 -0.71586852
 -0.86916075]
 [-0.30862192  0.72250438  0.92057462 -1.33667998 -0.26202255  2.51661148
 -0.99589349 -0.13820484 -0.71589105 -0.78921359 -0.96343165  2.63716726
 1.20824075]]
```

Baseline Model: Logistic Regression

In this section, we implement Logistic Regression as the baseline model.

This model serves as a simple linear benchmark against which more complex models will be compared.

```
In [119... from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score
```

```
In [120... # 1.Create the model
log_reg = LogisticRegression(max_iter=1000)

# 2.Train the model on scaled training data
log_reg.fit(X_train_scaled, y_train)

# 3.Make predictions on test data
y_pred_lr = log_reg.predict(X_test_scaled)
```

```

# 4. Calculate evaluation metrics
accuracy_lr = accuracy_score(y_test, y_pred_lr)
precision_lr = precision_score(y_test, y_pred_lr)
recall_lr = recall_score(y_test, y_pred_lr)
f1_lr = f1_score(y_test, y_pred_lr)

# 5. Print results
print("Logistic Regression Results:")
print("Accuracy:", accuracy_lr)
print("Precision:", precision_lr)
print("Recall:", recall_lr)
print("F1-score:", f1_lr)

# 6. Detailed report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_lr))

```

Logistic Regression Results:
 Accuracy: 0.8852459016393442
 Precision: 0.8787878787878788
 Recall: 0.90625
 F1-score: 0.8923076923076924

Classification Report:

	precision	recall	f1-score	support
0	0.89	0.86	0.88	29
1	0.88	0.91	0.89	32
accuracy			0.89	61
macro avg	0.89	0.88	0.88	61
weighted avg	0.89	0.89	0.89	61

Baseline Model Results

The Logistic Regression baseline model showed strong performance on the test set.

- Accuracy: 0.885
- Precision: 0.879
- Recall: 0.906
- F1-score: 0.892

The recall score is especially important in this medical classification task, as it indicates that the model successfully identified most patients with heart disease. Overall, Logistic Regression provides a strong and interpretable baseline for comparison with more complex models.

Additional Model 1: K-Nearest Neighbors

In this section, we implement the K-Nearest Neighbors (KNN) algorithm. KNN is a distance-based model that classifies a data point based on the majority class of its nearest neighbors.

```
In [121... from sklearn.neighbors import KNeighborsClassifier

# 1. Create the model
knn = KNeighborsClassifier(n_neighbors=5)

# 2. Train the model
knn.fit(X_train_scaled, y_train)

# 3. Make predictions
y_pred_knn = knn.predict(X_test_scaled)

# 4. Calculate metrics
accuracy_knn = accuracy_score(y_test, y_pred_knn)
precision_knn = precision_score(y_test, y_pred_knn)
recall_knn = recall_score(y_test, y_pred_knn)
f1_knn = f1_score(y_test, y_pred_knn)

# 5. Print results
print("KNN Results:")
print("Accuracy:", accuracy_knn)
print("Precision:", precision_knn)
print("Recall:", recall_knn)
print("F1-score:", f1_knn)

# 6. Classification report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_knn))
```

KNN Results:

Accuracy: 0.9180327868852459
Precision: 0.9354838709677419
Recall: 0.90625
F1-score: 0.9206349206349206

Classification Report:

	precision	recall	f1-score	support
0	0.90	0.93	0.92	29
1	0.94	0.91	0.92	32
accuracy			0.92	61
macro avg	0.92	0.92	0.92	61
weighted avg	0.92	0.92	0.92	61

KNN Model Results

- Accuracy: 0.918
- Precision: 0.935
- Recall: 0.906
- F1-score: 0.920

The model achieved higher accuracy and precision, indicating better overall classification performance. In particular, the higher precision suggests that KNN makes fewer false positive predictions.

Since KNN is a non-linear model, it is able to capture more complex patterns in the data compared to Logistic Regression. This suggests that the relationship between the features and heart disease is not purely linear.

Overall, KNN provides a strong improvement over the baseline model.

Additional Model 2: Support Vector Machine (SVM)

```
In [122... from sklearn.svm import SVC

# 1. Create the model
svm = SVC(kernel='rbf')

# 2. Train the model
svm.fit(X_train_scaled, y_train)

# 3. Make predictions
y_pred_svm = svm.predict(X_test_scaled)

# 4. Calculate metrics
accuracy_svm = accuracy_score(y_test, y_pred_svm)
precision_svm = precision_score(y_test, y_pred_svm)
recall_svm = recall_score(y_test, y_pred_svm)
f1_svm = f1_score(y_test, y_pred_svm)

# 5. Print results
print("SVM Results:")
print("Accuracy:", accuracy_svm)
print("Precision:", precision_svm)
print("Recall:", recall_svm)
print("F1-score:", f1_svm)

# 6. Classification report
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred_svm))
```

SVM Results:

Accuracy: 0.9016393442622951

Precision: 0.9333333333333333

Recall: 0.875

F1-score: 0.9032258064516129

Classification Report:

	precision	recall	f1-score	support
0	0.87	0.93	0.90	29
1	0.93	0.88	0.90	32
accuracy			0.90	61
macro avg	0.90	0.90	0.90	61
weighted avg	0.90	0.90	0.90	61

SVM Model Results

- Accuracy: 0.902

- Precision: 0.933
- Recall: 0.875
- F1-score: 0.903

The model achieved high precision, which means that when it predicts heart disease, it is often correct. This is valuable because it reduces the number of false positive cases.

However, its recall is slightly lower than that of Logistic Regression and KNN, which means it missed more actual positive cases. In a medical context, this can be an important limitation because failing to identify a patient with heart disease may have serious consequences.

Overall, SVM performed well and provided competitive results, but its lower recall makes it slightly less suitable than KNN for this specific task.

Model Comparison Table

In this section, we compare the performance of all models using a unified table of evaluation metrics.

```
In [123... import pandas as pd

# Create comparison table
comparison_df = pd.DataFrame({
    'Model': ['Logistic Regression', 'KNN', 'SVM'],
    'Accuracy': [accuracy_lr, accuracy_knn, accuracy_svm],
    'Precision': [precision_lr, precision_knn, precision_svm],
    'Recall': [recall_lr, recall_knn, recall_svm],
    'F1-score': [f1_lr, f1_knn, f1_svm]
})

comparison_df
```

```
Out [123... 
```

	Model	Accuracy	Precision	Recall	F1-score
0	Logistic Regression	0.885246	0.878788	0.90625	0.892308
1	KNN	0.918033	0.935484	0.90625	0.920635
2	SVM	0.901639	0.933333	0.87500	0.903226

Cross-Validation

In this section, we evaluate the stability and generalization of each model using 5-fold cross-validation.

Cross-validation helps ensure that the model performs consistently across different subsets of the data, rather than relying on a single train-test split.


```
In [124... from sklearn.model_selection import cross_val_score
import numpy as np
```

```
In [125... cv_lr = cross_val_score(log_reg, X_train_scaled, y_train, cv=5)

print("Logistic Regression CV Mean:", cv_lr.mean())
print("Logistic Regression CV Std:", cv_lr.std())
```

Logistic Regression CV Mean: 0.826530612244898
Logistic Regression CV Std: 0.03781863258152566

```
In [126... cv_knn = cross_val_score(knn, X_train_scaled, y_train, cv=5)

print("KNN CV Mean:", cv_knn.mean())
print("KNN CV Std:", cv_knn.std())
```

KNN CV Mean: 0.7852040816326531
KNN CV Std: 0.05386849392195397

```
In [127... cv_svm = cross_val_score(svm, X_train_scaled, y_train, cv=5)

print("SVM CV Mean:", cv_svm.mean())
print("SVM CV Std:", cv_svm.std())
```

SVM CV Mean: 0.8097789115646258
SVM CV Std: 0.025175812913056417

- 5-fold cross-validation was performed for each model to evaluate their stability and generalization performance.
- The mean cross-validation score represents the average model performance across different folds, while the standard deviation indicates how consistent the model is.
- Lower standard deviation suggests that the model performs reliably across different subsets of the data.

NOTE THAT:

Although KNN achieved the highest test accuracy, its lower cross-validation score and higher standard deviation suggest that it may be less stable and more sensitive to the training data.

In contrast, Logistic Regression and SVM demonstrated more consistent performance across folds.

Progress Log

Throughout this project, our team followed a structured approach to build and evaluate machine learning models for heart disease prediction.

Initially, we selected the Heart Disease dataset and performed exploratory data analysis (EDA) to understand the features and their relationships with the target

variable. During this stage, we identified that some features contained missing values, particularly in the "ca" and "thal" columns.

One of the main challenges we encountered was handling missing values, as some models such as Logistic Regression do not accept NaN values. We resolved this issue by applying median imputation, which is suitable for numerical medical data and robust to outliers.

We then transformed the target variable into a binary classification problem, grouping all non-zero values as indicating the presence of heart disease. This allowed us to apply standard classification algorithms.

Next, we implemented three models: Logistic Regression as a baseline, K-Nearest Neighbors (KNN), and Support Vector Machine (SVM). Each model was trained on scaled data and evaluated using accuracy, precision, recall, and F1-score.

We observed that KNN achieved the best test performance, while Logistic Regression and SVM provided more stable results based on cross-validation. This highlighted the trade-off between model performance and stability.

Overall, the project helped us understand the importance of preprocessing, model selection, and evaluation techniques in machine learning workflows. It also demonstrated how different models can behave differently on the same dataset.